

Phishing Detection Using Machine Learning Techniques

Critical Reproduction and Evaluation Study

Student: Saleh

ID: 212175111

Course: Data Science in Cyber

Source: Shahrivari, Darabi & Izadi (2020) — arXiv:2009.11116

Original repository: https://github.com/fafal-abnir/phishing_detection

1. SUMMARY OF THE SOURCE

The paper addresses binary classification of phishing versus legitimate websites. Given URL and website-level features, the goal is to predict whether a site is a phishing attempt designed to steal credentials or other sensitive information.

Phishing causes large financial and reputational damage. The authors cite FBI IC3 data (\$2.7B losses in 2018) and APWG trends showing sustained growth in phishing sites. Automated detection scales better than user training alone and can block malicious pages before users interact with them.

The proposed solution is to compare twelve supervised machine learning classifiers on a standard phishing feature dataset: Logistic Regression, Decision Tree, Random Forest, AdaBoost, KNN, SVM (four kernels), Neural Network, Gradient Boosting, and XGBoost. The best model is selected by accuracy, precision, recall, F1, and runtime.

The dataset is the UCI Phishing Websites dataset (Mohammad, Thabtah & McCluskey, 2015), with 11,055 instances, 30 pre-extracted features per URL/site, labeled -1 (phishing) or 1 (legitimate). In our copy, approximately 56% of samples are legitimate and 44% are phishing.

The methodology uses 10-fold cross-validation on the full dataset after global feature standardization. Default hyperparameters are used for all models, although the paper mentions tuning. Primary metrics are accuracy, precision, recall, F1, and train/test time. XGBoost is reported as the best model, with about 98.3% accuracy in Table III.

2. CRITICAL EVALUATION

The main claims made by the authors are that machine learning can effectively detect phishing websites from URL and structure features; that ensemble and boosting methods (Random Forest, XGBoost) outperform simpler classifiers; that XGBoost achieves the highest accuracy and a strong F1 score on this dataset; that feature selection and hyperparameter tuning were applied to obtain the best results; and that the comparative study is reproducible via the published GitHub repository.

These claims are only partially supported, and the gaps matter for how the paper should be read.

On model ranking, the evidence is reasonably strong. Our reproduction of `All_Classifair.ipynb` and our own pipeline both recover the same ordering: ensemble and boosting methods beat logistic regression and weak SVM kernels, and Random Forest and XGBoost reach about 97% accuracy, close to Table III. The claim that ensembles work well on this feature set is therefore supported. However, the absolute XGBoost gap (paper 98.3% versus our 97.2%) shows that reported numbers are sensitive to library versions, unseeded shuffling, and preprocessing details that the paper does not pin down. More importantly, the paper presents XGBoost as clearly best, but Random Forest and the neural network are within about one percentage point. That is not a decisive superiority claim; it is a small margin on a single static dataset.

On hyperparameter tuning and feature selection, the claims are not supported by the published code. The notebook uses default estimators for essentially all models. There is no grid search, no documented search space, and no ablation that isolates the effect of tuning. Feature selection is mentioned in the text but

not implemented as a controlled experiment. This is a serious mismatch between narrative and evidence: readers are led to believe that the reported scores reflect optimized models, when they largely reflect default sklearn and XGBoost settings on a strong pre-engineered feature set.

The evaluation methodology is only partially appropriate for a cybersecurity detection problem. Ten-fold cross-validation on a fixed tabular dataset is a reasonable baseline, but several design choices weaken the conclusions. First, global scaling before cross-validation leaks test-fold statistics into training, which can inflate scores for distance-based models. Second, stratification is not explicit in the authors' code, so fold class balance is not guaranteed. Third, there is no held-out temporal or out-of-distribution test set. Phishing tactics evolve; a model that scores 97% on a 2015-era UCI snapshot may degrade on modern URLs, and the paper provides no evidence about that risk. Fourth, the metric suite is incomplete for security. Accuracy, precision, recall, and F1 are reported, but the paper does not emphasize phishing as the positive class, does not report MCC, ROC-AUC, PR-AUC, or F2, and does not discuss threshold choice. In phishing detection, false negatives (missed attacks) are typically more costly than false positives (extra warnings). F2 and PR-AUC are therefore more informative than accuracy alone, and a default threshold of 0.5 is an operational decision, not a neutral default. Our threshold-tuning analysis shows that lowering the decision threshold on P(phishing) can cut missed phishing cases substantially, at the cost of more false alarms. The authors never examine this trade-off.

There is also a subtle but important label-orientation issue. The dataset encodes phishing as -1 and legitimate as 1. In sklearn, many metrics treat the greater label as the positive class by default, so "precision" and "recall" in an unconfigured pipeline can refer to legitimate sites rather than phishing. The paper does not clarify which class is treated as positive. That ambiguity makes the security interpretation of their precision and recall weaker than it appears.

Additional weaknesses include that roughly half of the paper is background material with limited experimental depth; features are pre-engineered by a third party and the authors neither extract nor justify them; there is no error analysis, confusion matrix discussion, or security interpretation of mistakes; Table II is mislabeled as classification results although it shows feature statistics; the neural network section refers to "30 hidden layers," which likely means 30

neurons; the repository is incomplete (missing dependencies, README typos, no random seeds); the dataset has no timestamps, so temporal validation is impossible; and high feature redundancy (12 pairs with Spearman $|r| \geq 0.7$) is never addressed. The paper also does not discuss concept drift, adversarial URL evasion, or how list-based and ML-based detectors should be combined in practice, even though the introduction surveys those approaches.

The conclusion that ensemble methods are strong baselines on this dataset is justified. The stronger implications — that XGBoost is clearly superior, that the pipeline is carefully tuned, and that the study is fully reproducible — are not justified. Overall, the work is a useful benchmark comparison on a public feature set, but it overstates methodological rigor and under-analyzes the security-relevant aspects of evaluation.

3. FEATURE ENGINEERING ANALYSIS

Feature engineering was performed only minimally in the original paper and repository. Features are pre-extracted by the UCI dataset authors as 30 URL/website attributes encoded as -1, 0, or 1. The authors apply only a global StandardScaler before cross-validation. No encoding, feature creation, or feature selection is documented.

The 30 features come from the UCI Phishing Websites dataset and include SSLfinal_State, URL length, Prefix_Suffix, domain age, and Page_Rank. The target variable is Result, where -1 means phishing and 1 means legitimate.

For transformations, the authors use `sklearn.preprocessing.scale()` on the full feature matrix before cross-validation. In our pipeline, we placed StandardScaler inside a sklearn Pipeline so scaling is fit on each training fold only, which avoids data leakage. No extra encoding was needed because the features are already numeric. We did not drop correlated features in this baseline, but EDA found 12 pairs with Spearman $|r| \geq 0.7$, including Favicon and popUpWidnow ($r = 0.94$).

Redundancy was detected through Spearman correlation and high-correlation pairs. Redundant URL-structure signals could be pruned (for example, keeping one of Favicon or popUpWidnow) or handled with regularization, which tree models partly do implicitly. The authors do not discuss redundancy.

The feature engineering is partially meaningful. The underlying UCI features capture real phishing indicators such as suspicious SSL, URL tricks, and domain metadata. The authors' only transformation, scaling, helps distance-based models such as SVM, KNN, and logistic regression, but applying it globally is methodologically weak. Tree-based models are less sensitive to scaling.

Additional features that could improve performance include fresh lexical URL tokens, certificate transparency logs, page content embeddings, reputation feeds, and temporal features such as domain age in days and time since certificate issue. The paper itself suggests adding more features in future work.

4. REPRODUCIBILITY ANALYSIS

The code can be executed successfully only after fixes. After installing missing packages (xgboost, lightgbm, catboost) and remapping labels from $\{-1, 1\}$ to $\{0, 1\}$ for XGBoost, the notebook runs end-to-end. The original repository does not document these steps.

Required files and dependencies are mostly available. The dataset dataset.csv is included in the authors' repository (11,055 rows, 30 features plus label). All_Classifair.ipynb and dataset_description.ipynb are provided. However, requirment.txt lists only numpy, scikit-learn, beautifulsoup4, whois, google, seaborn, and pandas, while omitting xgboost, lightgbm, and catboost, which are imported in the main notebook. The README also references "requirments.txt" (misspelled) instead of "requirment.txt".

Hidden preprocessing steps include applying global StandardScaler to all features before cross-validation through preprocessing.scale on the full matrix X, which is not emphasized in the paper and can leak fold statistics into training.

The dataframe is shuffled with `sklearn.utils.shuffle()` without a `random_state`, so runs are not exactly reproducible across sessions. Label encoding uses `phishing = -1` and `legitimate = 1`, but newer XGBoost versions require `{0, 1}` and fail without remapping.

Overall reproducibility is moderate. The same dataset and general pipeline reproduce results that are close to the paper (within about 1–2% accuracy for most models), but exact numbers differ because of library versions, unseeded shuffling, and environment fixes not included in the original repo. A reader cannot fully reproduce the paper from the README alone without troubleshooting.

5. EXPERIMENTAL RESULTS

We performed two sets of experiments. First, we re-ran `All_Classifair.ipynb` from the authors' GitHub repository using 10-fold cross-validation and the same default classifiers. Second, we built our own pipeline in `project.ipynb` using stratified 10-fold cross-validation, `random_state=42`, `StandardScaler` inside a `Pipeline`, three models (Logistic Regression, Random Forest, and XGBoost), and security-oriented metrics with phishing as the positive class.

For the reproduction run, modifications included installing `xgboost`, `lightgbm`, and `catboost`, and remapping labels for XGBoost. In our study, we added per-fold scaling, fixed seeds, `StratifiedKFold`, MCC, F2, ROC-AUC, PR-AUC (average precision), threshold tuning, and out-of-fold error analysis.

An important correction concerns ROC-AUC. An earlier version of our notebook reported ROC-AUC values near zero despite strong precision, recall, and F1. That was a label-orientation bug: `sklearn's roc_auc_score` treats the greater label as positive by default, so with labels `{-1, 1}` the positive class is legitimate, not phishing. Passing `P(phishing)` against that encoding inverted the AUC. We fixed this by encoding phishing as 1 and scoring `P(phishing)` against that label. Corrected ROC-AUC values are high and consistent with the other metrics.

Reproduction results compared to the paper (accuracy) were as follows:
Decision Tree 0.965 (paper 0.966), Random Forest 0.973 (paper 0.973), XGBoost 0.972 (paper 0.983), Logistic Regression 0.927 (paper 0.927), Neural Network 0.970 (paper 0.970), and SVM RBF 0.952 (paper 0.952).

Our pipeline results (10-fold cross-validation, phishing = positive class, default threshold 0.5) were:

Logistic Regression — accuracy 0.928, precision 0.929, recall 0.907, F1 0.918, F2 0.911, MCC 0.854, ROC-AUC 0.979, PR-AUC 0.977.

Random Forest — accuracy 0.971, precision 0.976, recall 0.959, F1 0.967, F2 0.962, MCC 0.942, ROC-AUC 0.996, PR-AUC 0.995.

XGBoost — accuracy 0.972, precision 0.975, recall 0.963, F1 0.969, F2 0.965, MCC 0.944, ROC-AUC 0.997, PR-AUC 0.996.

F2 and PR-AUC are especially relevant for phishing detection because they emphasize recall and ranking quality for the security-sensitive positive class. XGBoost remains best overall, but Random Forest is nearly identical on ranking metrics.

We also tuned the decision threshold on out-of-fold $P(\text{phishing})$ to maximize F2. At the F2-optimal thresholds, XGBoost uses threshold 0.15 (precision 0.916, recall 0.989, F2 0.973), reducing missed phishing from 182 to 56 compared with threshold 0.5, while increasing false positives from 123 to 442. Random Forest's best F2 threshold is 0.30 (recall 0.981, F2 0.975). This shows that the default 0.5 threshold is not security-optimal: a lower threshold catches more phishing at the cost of more alerts.

Error analysis on XGBoost at threshold 0.5 found 182 false negatives and 123 false positives. Missed phishing sites differed most on `SSLfinal_State`, `URL_of_Anchor`, and `web_traffic` compared with correctly detected phishing, suggesting that borderline SSL and URL cases drive many errors.

These results support the authors' ranking, with ensemble methods clearly outperforming logistic regression, but they also show that high accuracy still leaves hundreds of missed phishing URLs unless the threshold is tuned for recall.

6. CONCLUSIONS

The Shahrivari et al. study is a reproducible benchmark on the UCI Phishing Websites dataset rather than a novel detection method. Ensemble models such as Random Forest and XGBoost achieve about 97% accuracy, and our improved pipeline confirms this with proper per-fold scaling and phishing-focused metrics, including corrected ROC-AUC (about 0.997 for XGBoost), PR-AUC (about 0.996), and F2 (about 0.965 at threshold 0.5). Reproducibility is moderate: the code runs after fixes, but dependencies, seeds, and label encoding issues are not documented. At the default threshold, XGBoost still misses 182 phishing sites; threshold tuning for F2 reduces that to 56, showing that operational decisions matter as much as model choice.

We learned that reproducing a paper requires checking the repository, not only the PDF, because hidden preprocessing and missing packages matter. Scaling must be applied inside cross-validation to avoid leakage. Label orientation must be explicit: treating the wrong class as positive can invert ROC-AUC and misinterpret precision and recall. For cybersecurity classification, PR-AUC, F2, and threshold tuning are more informative than accuracy alone. EDA also revealed substantial feature redundancy that the authors never discuss.

Strengths of the proposed solution include a simple pipeline, fast training, strong baselines on a public dataset, comparison of multiple models, and open data and code. Weaknesses include an outdated feature set, minimal feature engineering, weak experimental rigor (no stratification, global scaling, no error analysis or threshold analysis in the original work), overstated claims about tuning, incomplete security metrics, and no discussion of deployment or concept drift.

Suggestions for future improvements include adding fresh features (URL tokens, certificates, reputation APIs, page content), using temporal or geographically held-out test sets, reporting confusion matrices, MCC, ROC-AUC, PR-AUC, and F2 with phishing as the explicit positive class, tuning thresholds for operational cost, addressing redundancy through feature selection or PCA, and comparing against modern phishing detectors while discussing adversarial URL evasion.

7. EXECUTIVE SUMMARY

This project critically evaluated and reproduced "Phishing Detection Using Machine Learning Techniques" (Shahrivari et al., 2020), which compares twelve machine learning classifiers on the UCI Phishing Websites dataset of about 11,000 URLs and 30 hand-crafted features.

We reproduced the authors' Jupyter notebook and built an improved analysis pipeline. Reproduction succeeded after installing undeclared dependencies and fixing XGBoost label encoding. Results broadly matched Table III, for example Random Forest at about 97.3% accuracy and XGBoost at about 97.2% compared with 98.3% in the paper. Our pipeline added stratified cross-validation, per-fold scaling, fixed random seeds, and security-oriented metrics: MCC, F2, ROC-AUC, PR-AUC, threshold tuning, and error analysis.

The paper's main claim that ensemble and boosting methods outperform simpler models is supported. However, critical weaknesses include global scaling before cross-validation (data leakage risk), no reproducibility seeds, overstated hyperparameter tuning, missing error analysis, incomplete security metrics, and no discussion of false negatives or decision thresholds. Feature engineering is minimal; the value lies in the UCI features, not in the authors' transformations.

XGBoost achieved the best overall performance in our study (97.2% accuracy, 96.3% phishing recall, F2 0.965, ROC-AUC 0.997, PR-AUC 0.996 at threshold 0.5). Tuning the threshold to maximize F2 raises recall to 0.989 and cuts missed phishing from 182 to 56, at the cost of more false positives. We recommend this benchmark for teaching and baseline comparison, not as a production-ready

detector without substantial feature updates, explicit positive-class metrics, and operational threshold design.

8. SUMMING IT UP

The problem addressed is automated detection of phishing websites using supervised learning on URL and website features.

The selected article is Shahrivari, Darabi & Izadi (2020), arXiv:2009.11116, with code at github.com/fafal-abnir/phishing_detection.

The dataset used is the UCI Phishing Websites dataset with 11,055 samples, 30 features, and a binary label (-1 for phishing, 1 for legitimate).

Our methodology reproduced the authors' 10-fold cross-validation benchmark and extended it with stratified cross-validation, Pipeline scaling, three models, MCC, F2, corrected ROC-AUC, PR-AUC, threshold tuning, a confusion matrix, and false negative/false positive error analysis.

The main findings of the reproduction study are that results closely match the paper's model ranking, with XGBoost and Random Forest leading, but reproducibility requires undeclared fixes and the original methodology has leakage, metric, and reporting gaps.

The authors' claims are supported regarding model ranking and the general effectiveness of ensemble methods, but not regarding full reproducibility from the README, rigorous tuning, feature selection, or security-complete evaluation.

The most important insights are that high accuracy can still leave hundreds of undetected phishing URLs at the default threshold; that PR-AUC, F2, and threshold tuning matter more than accuracy alone for phishing detection; that

label orientation must be explicit or ROC-AUC can be inverted; and that feature redundancy and stale features limit real-world applicability.

For similar problems, we recommend using this work as a baseline benchmark on the UCI dataset, but extending features, fixing cross-validation methodology, reporting phishing-positive metrics (including PR-AUC and F2), and tuning thresholds for operational FP/FN costs before trusting deployment.

In conclusion, the article provides a useful and mostly reproducible comparison study whose main conclusions about ensemble performance hold, but whose experimental rigor, metric design, and reproducibility documentation fall short of best practice for cybersecurity machine learning. Our analysis confirms the broad results while identifying methodological improvements essential for credible evaluation.